# Tydenso

Symbolic tensor algebra inside Typst

User manual · version 0.1.0

Build indexed tensors as readable Typst values, transform them with Idenso, and print them with Spenso's own notation.

Repository · MIT license · Spenso Python API · Idenso Python API

# Contents

1 Start with a contraction ... 3
1.1 Installation ... 3
1.2 How this corresponds to the Python API ... 3
2 Build tensor expressions ... 3
2.1 Representations and slots are data ... 3
2.2 Tensor names carry symmetry ... 5
2.3 Write tensor algebra as math ... 5
2.4 Give vectors mathematical labels ... 5
2.5 Build open and closed Spenso chains ... 6
3 Transform tensors ... 7
3.1 Contract a metric chain ... 7
3.2 Know which family to call ... 7
4 Display and inspect ... 8
4.1 Control tensor notation in Typst ... 8
4.2 Inspect the Atom as CBOR data ... 9
5 Work alongside Symbolica ... 9
6 API reference ... 10
6.1 Tensor construction ... 10
6.2 Expression construction ... 17
6.3 Spenso products and chains ... 19
6.4 Printing and inspection ... 21
6.5 Metric and index transformations ... 25
6.6 Dirac and color transformations ... 26
6.7 Advanced configuration ... 26
7 Compatibility and licensing ... 27
8 Acknowledgements ... 27

# 1 Start with a contraction

Tydenso is the tensor-focused companion to Symbolica. It has its own package, manual, notation, and WebAssembly engine. You do not need Symbolica to construct, simplify, inspect, or display a tensor expression.

The first example contracts a Minkowski metric with a vector. A representation is an ordinary Typst dictionary; `slot` turns an index label into another dictionary; `vector` creates a callable rank-one tensor name.

```
#let V = mink(4)
#let mu = slot(V, 1)
#let nu = slot(V, 2)
#let p = vector("p")

#let before = math($#metric(V, mu, nu) #p(nu)$)
#let after = contract(before)

$
  #to-typst(before)
  quad arrow.r.long
  #to-typst(after)
$
```

$$g^{\mu\nu}p^{\nu} \quad \longrightarrow p^{\mu}$$

Here `mink` turns the integer slots `1` and `2` into $\mu$ and $\nu$. The constructor calls already produce readable math. Their hidden, versioned metadata contains the exact Symbolica Atom, so `math` does not have to infer a tensor from its appearance. It returns Atom bytes for algebra; `to-typst` renders transformed results.

## 1.1 Installation

Until Tydenso is published separately, install the `tydenso/typst` directory from this repository as its own local package. On Linux, place or symlink it at:

```
~/.local/share/typst/packages/local/tydenso/0.1.0
```

Then import it independently:

```
#import "@local/tydenso:0.1.0": *
```

From a checkout, `#import "path/to/gammaloop/tydenso/typst/lib.typ": *` works as well. The package directory already contains its compressed engine and loader.

## 1.2 How this corresponds to the Python API

The public concepts follow Spenso's Python surface, adapted to what is natural in Typst. Python objects can overload calls; Typst dictionaries cannot, so index construction is the explicit `slot(V, 1)`. Tensor names are functions, which keeps the pleasant `p(mu)` spelling.

| Python | Typst |
|---|---|
| `V = Representation.mink(4)`<br>`mu = V("mu")`<br>`nu = V("nu")`<br>`p = TensorName("p")`<br>`expr = TensorName.g(mu, nu) * p(nu)` | `#let V = mink(4)`<br>`#let mu = slot(V, 1)`<br>`#let nu = slot(V, 2)`<br>`#let p = vector("p")`<br>`#let expr = math($#metric(V, mu, nu)`<br>`#p(nu)$)` |

Representations and slots stay transparent dictionaries. Symbols and tensor calls are visible Typst math with exact Atom metadata; completed algebraic expressions are Atom bytes.

# 2 Build tensor expressions

## 2.1 Representations and slots are data

Built-in constructors cover the representations initialized by Spenso and Idenso: `mink`, `euc`, `lor`, `bis`, `spf`, `cof`, `coad`, and `cos`. Use `representation` when a model introduces another representation.

Each built-in owns a conventional palette for integer slots:

| Constructor | Class | Palette | Base row |
|---|---|---|---|
| `mink` | inline metric | $\mu, \nu, \rho, \sigma$ | top |
| `euc` | self-dual | $i, j, k, l$ | top |
| `lor` | dualizable | $\mu, \nu, \rho, \sigma$ | top |
| `bis` | self-dual | $a, b, c, d$ | bottom |
| `spf` | dualizable | $\alpha, \beta, \gamma, \delta$ | top |
| `cof` | dualizable | $i, j, k, l$ | top |
| `coad` | self-dual | $a, b, c, d$ | top |
| `cos` | dualizable | $I, J, K, L$ | top |

Every palette starts at 1. After its fourth entry it repeats with a numeric subscript, so `slot(mink(4), 5)` displays as $\mu_1$. A dualizable representation uses the same palette for its dual; only the script row flips.

A custom representation can name its automatic indices. The palette repeats; each pass adds a subscript. Manually written indices use the same notation but keep their own symbolic identity.

```
#let M = representation(
  "M",
  4,
  namespace: "example",
  self-dual: true,
  indices: ($mu$, $nu$),
)
#let T = tensor("T", namespace: "example")
#let expression = T(
  slot(M, 1),       // mu
  slot(M, 2),       // nu
  slot(M, 3),       // mu_1
  slot(M, $rho_2$), // exactly the index written here
)

#let detailed = notation(with-dim: true)
$ #to-typst(expression, notation: detailed) $
```

$$T^{\mu^{M_4}\,\nu^{M_4}\,\mu_1^{M_4}\,\text{spenso_index_ca779a3e4c5dad55f6580b15266ddb96960cbdc323c05e4ac5685a4a4d7a7503}^{M_4}}$$

Here the qualified form distinguishes the indices as members of $M_4$. A custom representation without `indices` keeps numeric display. A string is a Symbolica identifier and uses Symbolica's native quoted Typst output; Typst math such as `$rho_2$` instead creates an opaque index symbol carrying exactly that display notation. Neither form consults the automatic palette.

```
#let color = cof(3)
#let i = slot(color, 1)
#let j-lower = slot(color, 2, dual: true)

#table(
  columns: (auto, 1fr),
  inset: 5pt,
  [representation], [#color.name],
  [dimension], [#color.dimension],
  [first index], [#i.index],
  [second index is dual], [#j-lower.dual],
)
```

| representation | cof |
|---|---|
| dimension | 3 |
| first index | 1 |
| second index is dual | true |

Because those values are dictionaries, document code can validate dimensions, generate index families in a loop, or attach its own metadata before asking the plugin to construct an Atom. `dual-representation` returns the paired representation; `metric`, `identity-tensor`, and `flat-tensor` accept either index labels or already constructed slots.

## 2.2 Tensor names carry symmetry

`tensor` mirrors the useful part of Spenso's `TensorName`: a name plus optional symmetric, antisymmetric, cyclic, or linear behavior. Symbolica applies those attributes while it constructs the function, not as a later cosmetic step.

```
#let V = euc(3)
#let mu-slot = slot(V, $mu$)
#let nu-slot = slot(V, $nu$)
#let F = tensor("F", antisymmetric: true)

#let cancellation = math($#F(mu-slot, nu-slot) + #F(nu-slot, mu-slot)$)
$ F^(mu nu) + F^(nu mu) = #to-typst(cancellation) $
```

$$F^{\mu\nu} + F^{\nu\mu} = 0$$

Only one of `symmetric`, `antisymmetric`, and `cycle-symmetric` may be true for one tensor name. The `linear` flag is independent.

## 2.3 Write tensor algebra as math

Tensor and vector constructors are Typst functions. Interpolate their calls in math with `#F(...)`; interpolate scalar symbols with `#mass`. Parsely reads the ordinary arithmetic, while each annotated value contributes its exact Atom.

```
#let V = mink("D")
#let mu = slot(V, 1)
#let nu = slot(V, 2)
#let p = vector("p")
#let mass = symbol("m", namespace: "model")

#let shell = math($#metric(V, mu, nu) #p(mu) #p(nu) - #mass^2$)
$ #to-typst(shell) $
```

$$-m^2 + g^{\mu\nu} p^{\mu} p^{\nu}$$

The structural `add`, `mul`, `neg`, `sub`, `div`, and `pow` functions remain useful for generated expressions. They accept Atom bytes, annotated content, numbers, slots, and representation dictionaries. Both paths construct the same Atom; neither reconstructs tensors from printed subscripts.

## 2.4 Give vectors mathematical labels

Raw Typst math in a tensor or vector argument describes how one atomic label is written. It can contain arithmetic, attachments, fractions, calls, accents, and the usual structural math elements. This is useful for momentum labels that should stay visually rich without becoming part of the tensor algebra.

```
#let V = mink(4)
#let p = vector("p")

#let routed = p($arrow(x + y)$, V)
#let algebraic = p(math($x + y$), V)

#table(
  columns: (auto, 1fr),
  inset: 5pt,
  [one display-defined label], [$ #to-typst(routed) $],
  [a genuine Symbolica sum], [$ #to-typst(algebraic) $],
)
```

| one display-defined label | $p_{\text{spenso_index_273ac41ef7f4b9090253dc5d55c415dd28e0bf50b44ba1dea81b671c61d4892d}}^{\square}$ |
|---|---|
| a genuine Symbolica sum | $p_{x+y}^{\square}$ |

The first call stores one opaque Symbolica symbol together with a portable, structured Typst display tree. The second call stores the actual sum $x + y$, so Symbolica may inspect or transform it. Symbolica and Rubi preserve the display attachment without needing to understand it; Tydenso restores it when the payload returns.

## 2.5 Build open and closed Spenso chains

The chain helpers construct Spenso's actual Atom heads; they are not drawing commands. Compact endpoints are ordinary rank-one tensors carrying a representation, while `gamma(mu)` supplies Spenso's `in` and `out` placeholders for a chain factor.

```
#let M = mink(4)
#let B = bis(4)
#let mu = slot(M, 1)
#let nu = slot(M, 2)

#let p = vector("p")
#let q = vector("q")
#let u = vector("u")
#let v = vector("v")

#let scalar = dot(p(1, M), q(2, M))
#let open = chain(
  u(1, B),
  v(2, B),
  gamma(mu),
  gamma(p(1, M)),
  gamma(nu),
)
#let closed = trace(
  B,
  cyclic(gamma(mu), gamma(p(1, M)), gamma(nu)),
)

$ #to-typst(scalar) $
$ #to-typst(open) $
$ #to-typst(closed) $
```

$$p_1 \cdot q_2$$

$$\langle u_1 | \left[\gamma^\mu \not{p}_1 \gamma^\nu\right]_{\bullet\bullet} | v_2 \rangle$$

$$\mathrm{Tr}\left(\gamma^\mu \not{p}_1 \gamma^\nu\right)$$

Use `gamma(a, b, mu)` when the bispinor slots are explicit: its arguments are the same bispinor, bispinor, Lorentz order used by the stored Atom. `chain` keeps an ordered open sequence; `cyclic` marks the factor list of a closed sequence; and `trace` combines that cycle with its representation. The standalone notation example also inspects the constructed trees and checks their exact Spenso shapes.

`math($gamma(...)$)` is raw Atom syntax, so it preserves the arguments exactly as written and does not infer a tensor signature. Write that form in the same `(bispinor, bispinor, Lorentz)` order, or interpolate `#gamma(a, b, mu)`.

Explicit slots are also valid chain endpoints; they render as endpoint scripts on the chain body, while compact vectors produce the named bra and ket ends.

A string and Typst math deliberately mean different things. `slot(B, "mu")` uses the semantic Symbolica identifier `mu`; Symbolica prints its multi-character name as the quoted Typst text `"mu"`. `slot(B, $mu$)` creates a different, opaque index whose attached notation prints as the Greek $\mu$. They are not interchangeable contraction labels.

```
#let B = bis(4)
#let q = vector("q")
#let identifier = slot(B, "mu")
#let notation = slot(B, $mu$)

#table(
  columns: (auto, 1fr),
  [semantic identifier `"mu"`], [$ #to-typst(q(identifier)) $],
  [Typst notation `$mu$`], [$ #to-typst(q(notation)) $],
)
```

| `semantic identifier "mu"` | $q_{\mathrm{mu}}$ |
|---|---|
| `Typst notation $mu$` | $q_{\mathrm{spenso_index_e1d577fb39631a5f98eb034ac005ee9bcca57eae8181461a93584f77d946ee85}}$ |

Use palette integers for a representation's standard indices, and use math content when the written notation itself defines a manual index. Plain `$mu$` works when the name is unbound. After `#let mu = ...`, Typst resolves `$mu$` to that binding; write `$std.sym.mu$` when a literal Greek symbol must remain unambiguous.

# 3 Transform tensors

Idenso provides the domain-specific transformations. The most common ones contract tensor indices and simplify gamma matrices and color structures. Index-wrapping functions are available for lower-level workflows.

## 3.1 Contract a metric chain

The next expression contains two metrics and one vector. Simplifying metrics eliminates both contracted dummy indices in one pass.

```
#let V = mink(4)
#let mu = slot(V, 1)
#let nu = slot(V, 2)
#let rho = slot(V, 3)
#let p = vector("p")

#let chain = mul(
  metric(V, mu, nu),
  metric(V, nu, rho),
  p(rho),
)
#let reduced = contract(chain)

$
  #to-typst(chain)
  quad arrow.r.long
  #to-typst(reduced)
$
```

$$g^{\mu\nu}g^{\nu\rho}p^{\rho} \quad \longrightarrow p^{\mu}$$

`list-dangling` returns the free indices as Atom payloads. Pass an element to `to-typst`, `to-string`, or `inspect` just like any other expression.

## 3.2 Know which family to call

| Family | Purpose |
|---|---|
| `contract` | Metric, Minkowski, and bispinor structure. |
| `simplify-gamma`, `dirac-adjoint` | Dirac chains and conjugation. |
| `simplify-color` | Color generators and structure constants. |
| `cook-function`, `cook-indices`, `wrap-dummies`, `wrap-indices` | Canonical index organization and explicit wrappers. |
| `to-dots` | Render compact metric products as dots; call `contract` first for explicit indices. |
| `undo-dots` | Open dots into symbolic indexed contractions without expanding the numerator. |

The mathematical conventions and supported identities track Idenso's API.

# 4 Display and inspect

## 4.1 Control tensor notation in Typst

`to-typst` returns math content. Tydenso lays out slots, tensor arguments, chains, dots, and traces with ordinary Typst functions, so notation can follow the document's own visual language. `to-string` remains available when a compact Spenso expression is useful for logs or debugging.

```
#let V = mink(4)
#let p = vector("p")
#let expression = p(1, slot(V, 1))

#let detailed = notation(with-dim: true, commas: true)

Rendered: $ #to-typst(expression, notation: detailed) $

Compact Spenso form:
#raw(to-string(expression), block: true)
```

Rendered:

$$p_{1}{}^{\mu^{\mathrm{mink}_4}}$$

Compact Spenso form:

```
p(1,αmu)
```

Upper and lower indices align in matching columns. `bis` uses the conventional bottom row; the other built-in base orientations use the top row. A dualizable representation puts its dual orientation on the opposite row, while a self-dual representation does not flip.

Compact vectors nested inside another tensor have three layouts. The default `"ports"` layout keeps their bra or ket outside the tensor and marks the contracted argument position with a hollow port. `"schoonschip"` puts the bold vector label directly into that aligned column. `"call"` groups upper entries before a semicolon and lower entries after it.

```
#let L = lor(4)
#let p = vector("p", namespace: "notation_layout")
#let A = tensor("A", namespace: "notation_layout")
#let expression = A(
  slot(L, 1),
  p(1, L),
  slot(L, 2),
  slot(L, 3, dual: true),
)

#let scripts = notation(tensor-layout: "schoonschip")
#let call = notation(tensor-layout: "call")

#grid(
  columns: 2,
  gutter: 0.8em,
  [aligned scripts], $ #to-typst(expression, notation: scripts) $,
  [row-grouped call], $ #to-typst(expression, notation: call) $,
)
```

aligned scripts $A^{\mu \boldsymbol{p}_1 \nu}{}_{\rho}$

row-grouped call $A(\mu,\boldsymbol{p}_1,\nu;\rho)$

The script layout preserves every original argument column. The call layout preserves order within each row; the exact Atom annotation retains the complete original order. A call containing only lower entries starts with a semicolon, as in $A(;\rho)$. Commas are on by default in this mode and can be disabled with `commas: false`.

The most common layout switches live directly on `notation`: `tensor-layout`, `with-dim`, `parens`, `commas`, `symbol-scripts`, `component-style`, `index-gap`, and `factor-gap`. Concrete components use $A(x,7)^{0,1}$ by default; `component-style: "array"` selects $A(x,7)[0,1]$. Ordinary parameters stay in parentheses in both styles, independently of abstract-index layout. `print-settings` configures the separate compact string returned by `to-string`.

Exact namespaced heads and complete calls can be restyled without changing the Atom. A call renderer receives its arguments as both structured nodes and already rendered math. Tydenso attaches the exact call after the renderer returns, so the result can still be interpolated into `math` losslessly.

```
#let x = symbol("x", namespace: "notation_example")
#let f = function("f", namespace: "notation_example")
#let expression = f(add(x, 1))

#let display = notation(
  heads: ("notation_example::x": $xi$),
  calls: (
    "notation_example::f": ctx => {
      let (argument,) = ctx.visual-arguments
      $cal(F)[#argument]$
    },
  ),
)

$ #to-typst(expression, notation: display) $
```

$$\mathcal{F}[1 + \xi]$$

Use `tags` for a named family and `classes` for Symbolica attributes such as `"antisymmetric"`. Tags are canonical namespaced identities—for example `"model::kinematic"`—so they remain unambiguous when expressions move between plugins. Document overrides win over Tydenso's defaults and over display labels carried by a payload.

### 4.2 Inspect the Atom as CBOR data

Atom bytes remain the lossless transformation and interchange format. Before a transformation, annotated constructor content carries those same bytes in metadata. `inspect` accepts either form and decodes it to recursive Typst data. Every node has a `kind`; function nodes also expose their full and short names, arguments, and symmetry flags.

```
#let V = euc(3)
#let A = tensor("A", symmetric: true)
#let expression = A(slot(V, 1), slot(V, 2))
#let tree = inspect(expression)

#table(
  columns: (auto, 1fr),
  inset: 5pt,
  [node kind], [#tree.kind],
  [function], [#tree.short-name],
  [arguments], [#tree.arguments.len()],
  [symmetric], [#tree.symmetric],
)
```

| node kind | function |
|---|---|
| function | A |
| arguments | 2 |
| symmetric | true |

> **Why keep both forms?** CBOR is convenient for layouts, diagnostics, and package-level APIs. Reconstructing algebra from that tree would lose Symbolica state and exact coefficient details, so transformations continue to consume Atom bytes.

## 5 Work alongside Symbolica

Tydenso and Symbolica share the same Atom payload. A package can construct and simplify tensors with Tydenso, then pass the result to Symbolica for a general algebraic operation. Custom representations keep the information Tydenso needs to use them again, including their index palette.

```
#import "@local/tydenso:0.1.0" as tensors
#import "@local/symbolica:0.1.0" as algebra
```

```
#let V = tensors.representation(
  "M",
  4,
  namespace: "model",
  self-dual: true,
  indices: ($mu$, $nu$),
)
#let p = tensors.vector("p")
#let expression = p(tensors.slot(V, 1))

#let expanded = algebra.expand(expression)
#tensors.to-typst(expanded)
```

Keep package versions aligned. Matrix payloads from Symbolica are a different format and are not Tydenso inputs.

# 6 API reference

The groups below cover the imported top-level API. `init` returns the same surface as a dictionary bound to a selected plugin module.

## 6.1 Tensor construction

### 6.1.1 tensor

Construct a named tensor function.

The result is callable with any mixture of slots, representations, scalar values, display math, and compatible Atom payloads. Raw Typst math in a non-slot argument is retained as one atomic display label; pass `math("$...$")` when that argument should instead be parsed as Symbolica algebra. Symmetry flags follow Spenso's `TensorName` model and are registered on the underlying Symbolica symbol.

```
#let V = mink(4)
#let Z = tensor("Z")
#to-typst(Z(slot(V, 1), slot(V, 2)))
```

$$Z^{\mu\nu}$$

**Parameters**

```
tensor(
  name: str,
  namespace: str,
  symmetric: bool,
  antisymmetric: bool,
  cycle-symmetric: bool,
  linear: bool,
  tags: array
) -> function
```

#### 6.1.1.0.1 name `str`

Tensor name.

#### 6.1.1.0.2 namespace `str`

Symbol namespace.

Default: `"spenso"`

#### 6.1.1.0.3 symmetric `bool`

Make the tensor symmetric under argument permutation.

Default: `false`

#### 6.1.1.0.4 antisymmetric `bool`

Make the tensor antisymmetric under argument permutation.

Default: `false`

#### 6.1.1.0.5 cycle-symmetric `bool`

Make the tensor symmetric under cyclic argument permutation.

Default: `false`

#### 6.1.1.0.6 linear `bool`

Mark the tensor function as linear.

Default: `false`

#### 6.1.1.0.7 tags `array`

Portable semantic labels retained in attached metadata.

Default: `()`

### 6.1.2 vector

Construct a named rank-one tensor function.

Non-slot arguments are rendered as symbol subscripts in Spenso's Typst mode, while the vector slot is rendered as an abstract index. Raw Typst math is preserved as the notation for one atomic label, so accents, fractions, attachments, and arithmetic can be written directly. Use `math($...$)` when the argument should remain an algebraic expression rather than a label.

```
#let V = mink(4)
#let p = vector("p")
#to-typst(p($arrow(x + y)$, V))
```




**Parameters**

```
vector(
  name: str,
  namespace: str,
  linear: bool,
  tags: array
) -> function
```

#### 6.1.2.0.1 name `str`

Vector name.

#### 6.1.2.0.2 namespace `str`

Symbol namespace.

Default: `"spenso"`

#### 6.1.2.0.3 linear `bool`

Mark the vector function as linear.

Default: `false`

#### 6.1.2.0.4 tags `array`

Portable semantic labels retained in attached metadata.

Default: `()`

### 6.1.3 symbol

Construct a symbolic scalar with exact Atom metadata.

**Parameters**

```
symbol(
  name: str,
  namespace: str,
  tags: array
) -> content
```

#### 6.1.3.0.1 name `str`

Symbol name.

#### 6.1.3.0.2 namespace `str`

Symbol namespace.

Default: `"spenso"`

#### 6.1.3.0.3 tags `array`

Portable semantic labels retained in attached metadata.

Default: `()`

### 6.1.4 function

Construct a callable symbolic function with exact Atom metadata.

This is distinct from `symbol`: a Typst content value is not callable. Calling the returned function annotates the complete function call.

**Parameters**

```
function(
  name: str,
```

```
  namespace: str,
  tags: array
) -> function
```

#### 6.1.4.0.1 name `str`

Function-head name.

#### 6.1.4.0.2 namespace `str`

Symbol namespace.

Default: `"spenso"`

#### 6.1.4.0.3 tags `array`

Portable semantic labels retained in attached metadata.

Default: `()`

### 6.1.5 representation

Describe a representation and attach its tensor-building methods.

The returned dictionary is inspectable Typst data. Its `slot`, `metric`, `identity`, `flat`, and `dual` fields are functions.

**Parameters**

```
representation(
  name: str,
  dimension: int str bytes,
  namespace: str,
  self-dual: bool,
  is-dual: bool,
  indices: array none,
  index-start: int,
  index-row: str none
) -> dictionary
```

#### 6.1.5.0.1 name `str`

Symbolic representation name.

#### 6.1.5.0.2 dimension `int` or `str` or `bytes`

Dimension, which may itself be symbolic.

#### 6.1.5.0.3 namespace `str`

Symbol namespace.

Default: `"spenso"`

#### 6.1.5.0.4 self-dual `bool`

Whether the representation is its own dual.

Default: `false`

#### 6.1.5.0.5 is-dual `bool`

Whether this value denotes the dual orientation of a non-self-dual representation.

Default: `false`

#### 6.1.5.0.6 indices `array` or `none`

Fixed cyclic display palette for automatic integer indices. Entries may be Typst math content, strings, or integers. `none` selects the canonical built-in palette when one exists, and numeric display otherwise.

Default: `none`

#### 6.1.5.0.7 index-start `int`

Integer represented by the first palette entry. Each wrap adds a numeric subscript, so `($mu$, $nu$)` maps 1, 2, 3 to $\mu$, $\nu$, $\mu_1$.

Default: `1`

#### 6.1.5.0.8 index-row `str` or `none`

Preferred Typst script row for the representation's base orientation. `none` selects bottom for the built-in `spenso::bis` representation and top for every other representation. A dualizable representation's dual orientation uses the opposite row.

Default: `none`

### 6.1.6 mink

Construct a Minkowski representation.

Integer slots cycle through `mu`, `nu`, `rho`, and `sigma`. This is a self-dual inline-metric representation whose indices print on the top row.

**Parameters**

```
mink(dimension) -> dictionary
```

### 6.1.7 euc

Construct a Euclidean representation.

Integer slots cycle through `i`, `j`, `k`, and `l`. This representation is self-dual and prints its indices on the top row.

**Parameters**

```
euc(dimension) -> dictionary
```

#### 6.1.8 lor

Construct a Lorentz representation.

Integer slots cycle through `mu`, `nu`, `rho`, and `sigma`. Base indices print on the top row and dual indices on the bottom row.

**Parameters**

`lor(dimension) -> dictionary`

#### 6.1.9 bis

Construct a bispinor representation.

Integer slots cycle through `a`, `b`, `c`, and `d`. The representation is self-dual and conventionally prints all of its indices on the bottom row.

**Parameters**

`bis(dimension) -> dictionary`

#### 6.1.10 spf

Construct a spin-fundamental representation.

Integer slots cycle through `alpha`, `beta`, `gamma`, and `delta`. Base indices print on the top row and dual indices on the bottom row.

**Parameters**

`spf(dimension) -> dictionary`

#### 6.1.11 cof

Construct a color-fundamental representation.

Integer slots cycle through `i`, `j`, `k`, and `l`. Base indices print on the top row and dual indices on the bottom row.

**Parameters**

`cof(dimension) -> dictionary`

#### 6.1.12 coad

Construct a color-adjoint representation.

Integer slots cycle through `a`, `b`, `c`, and `d`. This representation is self-dual and prints its indices on the top row.

**Parameters**

`coad(dimension) -> dictionary`

#### 6.1.13 cos

Construct a color-sextet representation.

Integer slots cycle through `I`, `J`, `K`, and `L`. Base indices print on the top row and dual indices on the bottom row.

#### Parameters

```
cos(dimension) -> dictionary
```

### 6.1.14 slot

Construct an indexed slot from a representation.

The result is an ordinary dictionary, so its representation, index, and variance remain visible to Typst before an expression is constructed.

#### Parameters

```
slot(
  representation: dictionary,
  index: int str bytes content,
  dual: bool none
) -> dictionary
```

##### 6.1.14.0.1 representation `dictionary`

Representation dictionary.

##### 6.1.14.0.2 index `int` or `str` or `bytes` or `content`

Abstract index label. Typst math content such as `$mu_1$` is retained as safe display metadata on a symbolic index. A string instead names an ordinary Symbolica identifier and uses Symbolica's native quoted Typst output. If a local variable shadows a math name, qualify the literal symbol, for example `$std.sym.mu$`.

##### 6.1.14.0.3 dual `bool` or `none`

Wrap the slot with Spenso's dual-index marker. `none` inherits the representation's `is-dual` field.

Default: `none`

### 6.1.15 metric

Construct the metric tensor for a representation.

Index labels are converted to slots automatically; existing slots pass through unchanged.

#### Parameters

```
metric(
  representation,
  first,
  second
) -> content
```

### 6.1.16 identity-tensor

Construct the identity tensor for a representation.

#### Parameters

```
identity-tensor(
  representation,
```

```
  first,
  second
) -> content
```

#### 6.1.17 flat-tensor

Construct Spenso's musical isomorphism tensor for a representation.

**Parameters**

```
flat-tensor(
  representation,
  first,
  second
) -> content
```

#### 6.1.18 dual-representation

Return the representation paired with this representation.

**Parameters**

```
dual-representation(representation) -> dictionary
```

### 6.2 Expression construction

#### 6.2.1 math

Parse Typst math into an exact Symbolica Atom payload.

Annotated values created by `symbol`, `function`, `tensor`, `vector`, and the named tensor constructors are imported from their exact Atom metadata. Use interpolation for callable bindings, for example `#F(mu, nu)`.

Unannotated calls are literal Atom syntax: `math($gamma(...)$)` neither infers a tensor signature nor rearranges its arguments. Write raw gamma atoms in canonical storage order, `gamma(bispinor, bispinor, lorentz)`, or use `#gamma(a, b, mu)` for the typed constructor.

**Parameters**

```
math(
  equation: content,
  grammar: dictionary none,
  namespace: str none
) -> bytes
```

##### 6.2.1.0.1 equation `content`

Math content, normally written as `$...$`.

##### 6.2.1.0.2 grammar `dictionary` or `none`

Parser grammar override. `none` uses the engine grammar.

Default: `none`

##### 6.2.1.0.3 namespace `str` or `none`

Namespace for unannotated parsed symbols.

Default: `none`

#### 6.2.2 atom

Convert a supported value or annotated math expression to Atom bytes.

**Parameters**

```
atom(value) -> bytes
```

#### 6.2.3 add

Add symbolic expressions or constructor values.

**Parameters**

```
add(..terms) -> bytes
```

#### 6.2.4 mul

Multiply symbolic expressions or constructor values.

**Parameters**

```
mul(..factors) -> bytes
```

#### 6.2.5 neg

Negate a symbolic expression or constructor value.

**Parameters**

```
neg(expression) -> bytes
```

#### 6.2.6 sub

Subtract two symbolic expressions or constructor values.

**Parameters**

```
sub(
  left,
  right
) -> bytes
```

#### 6.2.7 div

Divide two symbolic expressions or constructor values.

**Parameters**

```
div(
  numerator,
  denominator
) -> bytes
```

#### 6.2.8 pow

Raise a symbolic expression or constructor value to a power.

### Parameters

```
pow(
  base,
  exponent
) -> bytes
```

## 6.3 Spenso products and chains

### 6.3.1 dot

Construct Spenso's compact scalar product `dot(left, right)`.

Rank-one tensor calls with a representation and no explicit slot are the compact Schoonschip form. Thus `dot(p(M), q(M))` emits exactly `dot(p(mink(4)),q(mink(4)))` when `M = mink(4)`.

### Parameters

```
dot(
  left,
  right
) -> content
```

### 6.3.2 gamma

Construct an Idenso gamma tensor or a gamma chain factor.

With one argument this emits the chain-factor shorthand `gamma(in,out,lorentz)`. With all three arguments, the public call and Atom use the same storage order: `gamma(first,second,lorentz)`. Those explicit arguments must be slots of `bis(4)`, `bis(4)`, and `mink(d)` respectively; a Lorentz-first tuple is rejected.

```
#let M = mink(4)
#let B = bis(4)
#let mu = slot(M, 1)
#let a = slot(B, 1)
#let b = slot(B, 2)
#let factor = gamma(mu)
#let explicit = gamma(a, b, mu)
#to-typst(chain(a, b, factor))
```

$$\left[\gamma^{\mu}\right]_{ab}$$

### Parameters

```
gamma(..arguments) -> content
```

### 6.3.3 gamma0

Construct the built-in $\gamma_0$ spinor factor.

With no arguments, the hidden endpoints are Spenso's `in` and `out` markers. Pass two explicit spinor slots to display its indices.

### Parameters

```
gamma0(..endpoints) -> content
```

### 6.3.4 gamma5

Construct the built-in $\gamma_5$ spinor factor.

### Parameters

```
gamma5(..endpoints) -> content
```

### 6.3.5 projp

Construct the positive-chirality spinor projector.

#### Parameters

```
projp(..endpoints) -> content
```

### 6.3.6 projm

Construct the negative-chirality spinor projector.

#### Parameters

```
projm(..endpoints) -> content
```

### 6.3.7 chain

Construct an open Spenso chain.

`start` and `end` may be explicit slots or compact rank-one tensors. For example, if `u = vector("u")` and `B = bis(4)`, then `u(B)` is the actual compact endpoint Atom `u(bis(4))`; no `bra` or `ket` head is introduced. Factors such as `gamma(mu)` already contain Spenso's `in` and `out` placeholders.

#### Parameters

```
chain(
  start,
  end,
  ..factors
) -> content
```

### 6.3.8 cyclic

Construct Spenso's inert cycle-symmetric projector.

This emits the actual Atom `cyclic(factors...)`.

#### Parameters

```
cyclic(..factors) -> content
```

### 6.3.9 trace

Construct a canonical closed Spenso chain.

A non-empty call accepts one explicit `cyclic(...)` payload and therefore emits exactly `trace(rep,cyclic(factors...))`. Passing a raw factor list is rejected instead of being silently rewritten. An empty call emits `trace(rep)`.

```
#let M = mink(4)
#let B = bis(4)
#let mu = slot(M, 1)
#let nu = slot(M, 2)
#to-typst(trace(B, cyclic(gamma(mu),
gamma(nu))))
```

$$\mathrm{Tr}\left(\gamma^{\mu}\,\gamma^{\nu}\right)$$

### Parameters

```
trace(
  representation,
  ..factors
) -> content
```

## 6.4 Printing and inspection

### 6.4.1 notation

Build Tydenso's document-side notation.

Exact `heads` and `calls` customize one namespaced Symbolica head or full call. `tags` and `classes` customize a family. Full-call renderers receive the complete arguments through their context; exact Atom metadata is added after they return.

### Parameters

```
notation(
  settings: dictionary,
  tensor-layout: str none,
  component-style: str none,
  with-dim: bool none,
  parens: bool none,
  commas: bool none,
  symbol-scripts: bool none,
  index-gap: length none,
  factor-gap: length none,
  heads: dictionary,
  calls: dictionary,
  tags: dictionary,
  classes: dictionary
) -> dictionary
```

#### 6.4.1.0.1 settings `dictionary`

Reusable notation settings. Directly named options override this layer.

Default: `(:)`

#### 6.4.1.0.2 tensor-layout `str` or `none`

Tensor layout: `"ports"` keeps the bra/ket port notation, `"schoonschip"` places bold compact vectors in aligned script columns, and `"call"` groups top and bottom rows around a semicolon.

Default: none

#### 6.4.1.0.3 component-style `str` or `none`

Concrete coordinates: `"superscript"` gives A(x,7)^{0,1}; `"array"` gives A(x,7)[0,1].

Default: none

#### 6.4.1.0.4 with-dim `bool` or `none`

Show each representation and dimension on its indices.

Default: none

#### 6.4.1.0.5 parens `bool` or `none`

Enclose chain bodies in brackets.

Default: `none`

#### 6.4.1.0.6 commas `bool` or `none`

Separate ordinary tensor arguments with commas.

Default: `none`

#### 6.4.1.0.7 symbol-scripts `bool` or `none`

Place non-index arguments in tensor scripts when possible.

Default: `none`

#### 6.4.1.0.8 index-gap `length` or `none`

Space between adjacent index columns.

Default: `none`

#### 6.4.1.0.9 factor-gap `length` or `none`

Space between factors in a chain or trace.

Default: `none`

#### 6.4.1.0.10 heads `dictionary`

Exact namespaced head renderers or content.

Default: `(:)`

#### 6.4.1.0.11 calls `dictionary`

Exact namespaced full-call renderers.

Default: `(:)`

#### 6.4.1.0.12 tags `dictionary`

Symbolica-tag renderer fallbacks.

Default: `(:)`

#### 6.4.1.0.13 classes `dictionary`

Attribute/class renderer fallbacks.

Default: `(:)`

### 6.4.2 merge-notation

Merge notation layers from least to most specific.

**Parameters**

```
merge-notation(..layers) -> dictionary
```

### 6.4.3 to-typst

Print and evaluate an expression as Typst math content.

The plugin exports a generic Atom tree. Tydenso's pure Typst notation lays out tensors, indices, gamma matrices, dots, chains, and traces. Generic algebra stays structurally visible; a customized full call receives one exact metadata annotation after its renderer returns.

**Parameters**

```
to-typst(
  expression,
  settings: dictionary none,
  notation: dictionary none,
  block: bool
) -> content
```

#### 6.4.3.0.1 expression

Atom payload or constructor value.

#### 6.4.3.0.2 settings `dictionary` or `none`

Compatibility shorthand for a default notation with these settings. Cannot be combined with `notation`.

Default: `none`

#### 6.4.3.0.3 notation `dictionary` or `none`

Per-call notation. `none` uses the engine default.

Default: `none`

#### 6.4.3.0.4 block `bool`

Display the result as a block equation.

Default: `false`

### 6.4.4 print-settings

Configure Spenso's string printer.

These settings are primarily for `to-string`. Typst math layout is more directly configured through `notation`.

**Parameters**

```
print-settings(
  preset: str,
  with-dim: bool none,
  parens: bool none,
```

```
  commas: bool none,
  index-subscripts: bool none,
  symbol-scripts: bool none
) -> dictionary
```

#### 6.4.4.0.1 preset `str`

Base preset: `"typst"` or `"compact"`.

Default: `"typst"`

#### 6.4.4.0.2 with-dim `bool` or `none`

Include representation dimensions in printed slots.

Default: `none`

#### 6.4.4.0.3 parens `bool` or `none`

Use parentheses around tensor arguments.

Default: `none`

#### 6.4.4.0.4 commas `bool` or `none`

Separate tensor arguments with commas.

Default: `none`

#### 6.4.4.0.5 index-subscripts `bool` or `none`

Render indices as subscripts where supported.

Default: `none`

#### 6.4.4.0.6 symbol-scripts `bool` or `none`

Render non-slot tensor arguments as symbol scripts where supported.

Default: `none`

### 6.4.5 to-string

Print an expression in Spenso's compact Symbolica notation.

**Parameters**

```
to-string(
  expression,
  settings: dictionary
) -> str
```

#### 6.4.5.0.1 expression

Atom payload or constructor value.

##### 6.4.5.0.2 settings `dictionary`

Settings created by `print-settings`.

Default: `_print-settings(preset: "compact")`

### 6.4.6 inspect

Decode an Atom payload into a recursive CBOR expression tree.

The result uses `kind` values such as `symbol`, `number`, `function`, `sum`, `product`, and `power`. Function nodes include their name, arguments, and symmetry flags. This view is ordinary Typst data; transforming the expression still uses its lossless Atom payload.

#### Parameters

`inspect(expression) -> dictionary`

## 6.5 Metric and index transformations

### 6.5.1 cook-function

Cook a tensor function into Idenso's canonical structure.

#### Parameters

`cook-function(expression) -> bytes`

### 6.5.2 cook-indices

Cook indices into Idenso's canonical structure.

#### Parameters

`cook-indices(expression) -> bytes`

### 6.5.3 list-dangling

Return the dangling indices as compatible Atom payloads.

#### Parameters

`list-dangling(expression) -> array`

### 6.5.4 contract

Contract and simplify metric tensors.

#### Parameters

`contract(expression) -> bytes`

### 6.5.5 to-dots

Render compact metric products as dots; use `contract` for indexed contractions.

#### Parameters

`to-dots(expression) -> bytes`

#### 6.5.6 undo-dots

Open dots into symbolic indexed contractions without expanding the numerator.

**Parameters**

```
undo-dots(expression) -> bytes
```

#### 6.5.7 wrap-dummies

Wrap dummy indices under the given header symbol.

**Parameters**

```
wrap-dummies(
  expression,
  header
) -> bytes
```

#### 6.5.8 wrap-indices

Wrap all indices under the given header symbol.

**Parameters**

```
wrap-indices(
  expression,
  header
) -> bytes
```

### 6.6 Dirac and color transformations

#### 6.6.1 dirac-adjoint

Take the Dirac adjoint of a tensor expression.

**Parameters**

```
dirac-adjoint(expression) -> bytes
```

#### 6.6.2 simplify-color

Simplify color tensors.

**Parameters**

```
simplify-color(expression) -> bytes
```

#### 6.6.3 simplify-gamma

Simplify gamma-matrix expressions.

**Parameters**

```
simplify-gamma(expression) -> bytes
```

### 6.7 Advanced configuration

#### 6.7.1 init

Create an independent Tydenso API.

The returned dictionary contains tensor constructors, document-side math rendering, CBOR inspection, and Idenso transformations. Use a custom `source` only when developing another compatible plugin; ordinary documents can use the imported top-level functions.

```
#let tensors = init()
#let V = (tensors.mink)(4)
#let p = (tensors.vector)("p")
#(tensors.to-typst)(p((tensors.slot)(V, 1)))
```

$p^{\mu}$

**Parameters**

```
init(
  namespace: str,
  source: str bytes none,
  grammar: dictionary,
  notation: dictionary
) -> dictionary
```

#### 6.7.1.0.1 namespace `str`

Default namespace for symbols parsed from Typst math.

Default: `"spenso"`

#### 6.7.1.0.2 source `str` or `bytes` or `none`

WebAssembly plugin path or uncompressed module bytes. `none` selects the bundled compressed Tydenso engine.

Default: `none`

#### 6.7.1.0.3 grammar `dictionary`

Parser grammar used by `math` unless it receives an explicit override.

Default: `_default-grammar`

#### 6.7.1.0.4 notation `dictionary`

Default document-side tensor notation used by constructors and `to-typst`.

Default: `tensor-notation.default-notation()`

# 7 Compatibility and licensing

This manual describes Tydenso 0.1.0 and Typst 0.15 or newer. Tydenso's original source is released under the MIT License. The bundled engine also contains Spenso, Idenso, and Symbolica; their own upstream terms continue to apply. In particular, the MIT license for this interface does not relicense Symbolica. Read Symbolica's license before redistributing or deploying the WebAssembly bundle.

# 8 Acknowledgements

Tydenso would not exist without Symbolica, the exact algebra engine beneath its Atom representation. Thank you to Symbolica's contributors, and to the GammaLoop contributors for Spenso and Idenso, whose tensor model, printers, and identities define this package's mathematics.

Thanks also to Tidy, which powers the worked examples and generated reference, and to Parsely, which parses Typst math while preserving the semantic Atom metadata.